Figure 3.3 Traditionally, applications are packaged as WARs and require a server to be available in the execution environment to run. Cloud native applications are packaged as JARs, are self-contained, and use an embedded server.

The embedded servers used for cloud native applications usually comprise a web server component and an execution context to make a Java web application interact with the web server. For example, Tomcat contains a web server component (Coyote) and an execution context based on the Java Servlet API, usually called the Servlet container (Catalina). I will use *web server* and *Servlet container* interchangeably. On the other hand, application servers are not recommended for cloud native applications.

In the previous chapter, when generating the Catalog Service project, we chose the JAR packaging option. We then ran the application using the `bootRun` Gradle task. That is a convenient way to build a project and run it as a standalone application during development. But now that you know more about the embedded server and JAR packaging, I'll show you another way.

First, let's package the application as a JAR file. Open a Terminal window, navigate to the root folder of the Catalog Service project (catalog-service), and run the following command.

```
$ ./gradlew bootJar
```

The `bootJar` Gradle task compiles the code and packages the application as a JAR file. By default, the JAR is generated in the build/libs folder. You should get an executable JAR file named catalog-service-0.0.1-SNAPSHOT.jar. Once you get the JAR artifact, you can go ahead and run it like any standard Java application.